

算法导论期末

Ch1 引言：某些典型的问题

稳定匹配

定义

不存在 (m, w) 和 (m', w') ，其中 m 更偏爱 w' ， w' 更偏爱 m

伪代码

```
初始化所有的男人 $m \in M$ 和女人 $w \in W$ 都是自由的
While 存在男人 $m$ 是自由的，且还没对所有女人求过婚
    If  $w$ 是自由的 then
         $(m, w)$ 约会
    Else 设 $w$ 当前在与 $m'$ 约会
        If  $w$ 更偏爱 $m'$  then
             $m$ 保持自由
        Else  $w$ 更偏爱 $m$ ， $m'$ 变成自由， $(m, w)$ 约会
    Endif
Endif
Endwhile
输出约会对象
```

算法结论

定理：一次G-S算法返回一个集合 S ，集合 S 是一个稳定匹配

算法证明

命题1：对于任意女人 w ，从接受第一次求婚开始就处于约会状态，她的对象会越来越好

取消匹配的时候她会选择更好的对象

命题2：对于任意男人 m ，他的对象越来越差

每次取消匹配会让他选择更差的对象

命题3：G-S算法中的while循环最多执行次数为 n^2

while循环相当于对男人的优先表进行遍历，男人的优先表一共有 n^2 个元素，所以while循环最多执行 n^2 次

命题4：如果 m 在算法执行的某点是自由的，那么一定存在一个他还没有求婚的对象

假设 m 对所有女人求过婚，那么根据命题1，每个女人都处于约会状态，也就是所有的男人都处于约会状态，与 m 不处于约会状态矛盾

命题5：终止时返回的集合 S 是一个完美匹配（即所有人都有匹配）

假设终止时有自由的男人，那么他已经像所有女人都求婚过，根据命题4，如果他对所有女人都求婚过，那么他一定不是自由的，与他是自由的矛盾

定理：一次G-S算法返回一个集合 S ，集合 S 是一个稳定匹配

假设集合 S 中有不稳定因素 (m, w) 和 (m', w') ，那么他们就有如下性质： m 更偏爱 w' ， w' 更偏爱 m 。根据算法的执行规则， m 最后一次求婚是向 w 求婚。如果在这个时间点前， m 向 w' 求过婚，那么 w' 当时的对象应该比 m 好，而且应该越来越好，但是最后 m' 比 m 差，矛盾；如果 m 没有向 w' 求过婚，那么 w 的优先级比 w' 高，矛盾。

综上，集合 S 中不存在不稳定因素，集合 S 是一个稳定匹配

稳定匹配的推广

公平性

稳定匹配对于性别是不公平的，如果男人求婚，那么最后是按照男人的优先表进行匹配，女人不一定能获得最优的对象；如果女人求婚，男人不一定能获得最优的对象。

执行顺序对匹配结果的影响

算法对每个男人的执行顺序对最后的结果没有影响，即

定理：G-S算法的每次执行都得到相同的集合 S^*

证明过程中用到命题

命题：在稳定匹配 S^* 中每个女人都与他最差的有效伴侣配对

五个典型问题

- 区间调度
- 带权区间调度
- 二分匹配
- 独立集
- 竞争便利店选址问题

Ch2 算法分析基础

多项式时间算法

存在绝对常数 $c > 0, d > 0$ ，是的对于每个规模为 N 的输入实例，他的运算时间不超过 cN^d 个基本运算步

渐近的上界

若存在常数 $c > 0$ 和 $n_0 \geq 0$ ，使得对于所有的 $n \geq n_0$ 有 $T(n) \leq c \cdot f(n)$ ，那么 $T(n) = O(f(n))$ ， f 是 $T(n)$ 的渐近上界

渐近的下界

若存在常数 $\epsilon > 0$ 和 $n_0 \geq 0$ ，使得对于所有的 $n \geq n_0$ 有 $T(n) \geq \epsilon \cdot f(n)$ ，那么 $T(n) = \Omega(f(n))$ ， f 是 $T(n)$ 的渐近下界

渐近的紧的界

如果我们证明 $T(n)$ 既是 $O(f(n))$ 的, 也是 $\Omega(f(n))$ 的, 那么 $T(n) = \Theta(f(n))$, $f(n)$ 是 $T(n)$ 的一个渐近的紧的界

命题: 设 f 和 g 是两个函数,

$$\lim_{g \rightarrow \infty} \frac{f(n)}{g(n)}$$

存在, 并且等于某个常数 $c > 0$, 那么 $f(n) = \Theta(g(n))$.

复杂度的性质

如果 $f = O(g)$ 且 $g = O(h)$, 那么 $f = O(h)$.

如果 $f = \Omega(g)$ 且 $g = \Omega(h)$, 那么 $f = \Omega(h)$.

如果 $f = \Theta(g)$ 且 $g = \Theta(h)$, 那么 $f = \Theta(h)$.

如果 $f = O(h)$ 且 $g = O(h)$, 那么 $f + g = O(h)$.

如果 $f = \Omega(h)$ 且 $g = \Omega(h)$, 那么 $f + g = \Omega(h)$.

如果 $f = \Theta(h)$ 且 $g = \Theta(h)$, 那么 $f + g = \Theta(h)$.

令 k 是固定常数, $f_1, f_2, \dots, f_k$ 和 h 是函数, 且对所有的 i , $f_i = O(h)$, 那么 $f_1 + f_2 + \dots + f_k = O(h)$.

若 $f_n = a_0 + a_1n + \dots + a_d n^d$, $a_d > 0$, 那么 $f = \Theta(n^d)$.

对于任何正整数, $O(\log_a n) = O(\log_b n)$.

对于 $x > 0$, $\log n = O(n^x)$, 即对数函数增长比指数函数慢

对于每个 $r > 1$ 和每个 $d > 0$, $n^d = O(r^n)$, 即多项式增长比指数函数慢

按时间分类

1. 亚线性时间 $O(\log n)$

- 二分查找: 从排好序的数组中查找一个数

2. 线性时间

- 计算 n 个数中的最大值
- 归并两个排好序的数表

3. $O(n \log n)$ 时间

- 快速排序
- FFT (快速傅里叶变换)
- 归并排序, 堆排序

4. 平方时间

- 遍历求解最近点对, 实际上可以优化

5. 立方时间

- n 个集合中不相交的集合

6. $O(n^k)$ 时间

- n 个节点的图是否有大小为 k 的独立集

7. 指数时间

- 找最大规模的独立集, $O(n^2 2^n)$

8. $n!$ 时间

- 穷举法求TSP问题, $(n-1)!$

Ch3 图

命题: 每棵 n 个节点的树恰好有 $n-1$ 条边

定理: 设 G 是具有 n 个节点的无向图, 下面任意两个语句都可以推出第三个语句.

(i) G 是连通的.

(ii) G 不包含一个圈.

(iii) G 有 $n-1$ 条边.

BFS算法

伪代码

BFS(s):

置Discovered[s]=true, 其他 v , 置Discovered[v]=false

初始化 $L[0]$, 单个元素 s 构成

置层计数器 $i=0$

置BFS树 T =空集

While $L[i]$ 非空

 初始化一个空表 $L[i+1]$

 For 每个结点 $L[i]$ 中结点 u

 考虑每条关联到 u 的边 (u, v)

 If Discovered[v]=false then

 置 Discovered[v]=true

 把边 (u, v) 加到树 T 上

 把 v 加到表 $L[i+1]$

 Endif

 Endfor

 层计数器加1

Endwhile

性质

定理: 如果图是以邻接表给出, BFS算法的时间复杂度为 $O(m+n)$

定理: BFS算法产生的层 L_i 就是到远点 s 距离为 i 的顶点的集合. 存在一条 s 到 t 的路径当且仅当 t 出现在某一层中.

定理: 设 T 是图 $G(V, E)$ 的一颗宽度优先搜索树, (x, y) 是 G 中的一条边, 那么 x, y 所属的层数至多相差1.

DFS算法

DFS(s)

初始化S为具有一个元素s的栈

While S 非空

从S中取出一个节点u

If Explored[u]=false then

置Explored[u]=true

For每条与u关联的边(u,v)

把v加到栈S

Endfor

EndIf

EndWhile

定理：如果图是以邻接表给出，BFS算法的时间复杂度为 $O(m+n)$

连通

BFS算法发现的是从始点s可达的结点。把这个集合R看作G包含s的连通分支。

定理：对无向图中的任两个结点s与t，他们的连通分支或者相等，或者不相交。

找出所有连通分支的集合的时间复杂度为 $O(m+n)$

有向图中的连通

在有向图中，有如下定义

u和v是相互可达的，如果彼此之间存在到达对方的路径。

如果图中的每对结点是相互可达的，那么此图是强连通的。

强连通性

引理：设s是图G中的任意一个结点，G是强连通的，当且仅当图中的每个结点能够与s相互可达。

存在 $O(m+n)$ 的有效算法判别图G是否强连通。

二部图

定义

二部图是一个图，其中节点集可以划分为x与y，每条边的一端在x中，另一端在y中。

二部图性质

定理：如果一个图是二部图，那么他不可能包含一个奇圈。

定理：设 G 是一个连通图， $L_0, \dots, L_k$ 是从顶点 s 由BFS算法生成的层。那么下面两件事一定恰好成立其一：

1. G 中没有边与同一层的两个结点相交。这种情况下 G 是二部图，其中偶数层的结点可以着红色，奇数层结点可以着蓝色。
2. G 有一条边与同一层的两个结点相交。此种情况下存在一个奇圈，不可能是二部图。

引理：图 G 是二部图当且仅当图中没有奇圈。

有向无圈图(DAG)与拓扑排序

定理：如果 G 有一个拓扑排序，那么 G 是一个DAG。

定理：如果 G 是一个DAG，那么 G 有一个拓扑排序。

Ch4 贪心算法

区间调度

按照截止时间排序，从截止时间最小的需求开始依次选择相容的需求

证明

假设 O 是一个最优区间集合，需要证明贪心算法生成的集合 $|A| = |O|$

设 $i_1, i_2, \dots, i_k$ 是 A 中的需求，并按照加入到 A 的次序排列， $|A| = k$ 。类似的， $j_1, j_2, \dots, j_m$ 是 O 中的需求。目的是为了证明 $k=m$

命题1： A 是一个相容的需求集

命题2：对所有的指标 $r \leq k$ ，我们有 $f(i_r) \leq f(j_r)$ 。

可以使用数学归纳法证明命题2。

定理：贪心算法返回一个最优的集合 A

排序的时间复杂度为 $O(n \log n)$ ，查找到时间复杂度为 $O(n)$ ，所以该算法的时间复杂度为 $O(n \log n)$

推广：区间划分问题

深度：通过时间线上任意一点的最大区间数

命题1：在任何区间划分的实例中，资源数必须至少是区间集合的深度

使用如下算法给他们分配标签

```
任意打破区间的并列,根据开始时间对它们排序
设  $I_1, I_2, \dots, I_n$  表示按这个次序排列的区间
For  $j=1, 2, 3, \dots, n$ 
  For 每个按照上述次序领先于  $I_j$  并且与它重叠的区间  $I_i$ 
    从对  $I_j$  的考虑中排除  $I_i$  的标签
  Endfor
  If 在  $\{1, 2, \dots, d\}$  中存在任何还没有被排除的标签 then
    分配一个没排除的标签给  $I_j$ 
  Else
    保留  $I_j$  不分配标签
  Endif
Endfor
```

命题2: 如果我们使用上述算法, 每个区间将被分配一个标签, 且没有两个重叠的区间接受相同的标签。

定理: 上述贪心算法使用与区间集合深度等量的资源为每个区间安排一个资源, 这就是所需资源的最优数量。

算法时间复杂度为 $O(n \log n)$

最小延迟调度

按照截止时间排序, 没有时间空隙, 依次完成

最优高速缓存

最远将来规则: 收回在最远的将来被需要的那个项

最近最少原则: 收回最久以前被访问的页面

图的最短路径-Dijkstra算法

Dijkstra算法 (G, l)

设S是被探查的结点集合

对每个S中的u, 存储一个d(u);

初始S={s} 且d(s)=0

While S!=V

选择一个结点不在S中的结点v, 使得从S到v至少
有一条边连接并且

$d'(v) = \min_{e=(u,v), u \in S} d(u) + l_e$ 最小

将v加入S并且定义d(v)=d'(v)

Endwhile

Q1

n个顶点, m条边, 时间复杂度为 $O(mn)$

如果采取优先队列, 时间复杂度为 $O(m \log n)$

最小生成树

Prim算法: 保持探索过的结点, 对每一个没有探索过的结点, 将最小费用的边加入, 采用优先队列, 时间复杂度为 $O(m \log n)$

Kruskal算法: 按照边费用递增的顺序选择加入, 但不围成圈, 采取并查集的数据结构, 时间复杂度为 $O(m \log n)$

反向删除算法: 删除费用最高的边, 保持连通性

聚类

运行Kruskal算法, 删除最贵的k-1条边

huffman编码

采取优先队列, 运行时间为 $O(k \log k)$

Ch5 分治策略

归并排序 递推时间复杂度

每次排序时需要维护一个临时数组, 需要 $O(n)$ 次比较, 所以时间递推关系为

$$T_n \leq \begin{cases} 2T(n/2) + cn, & n > 2 \\ c, & n = 2 \end{cases}$$

时间复杂度为 $O(n \log n)$

对于一般递推关系式

$$T_n \leq \begin{cases} qT(n/2) + cn, & n > 2 \\ c, & n = 2 \end{cases}$$

时间复杂度为 $O(n^{\log_2 q})$

计数逆序

Merge-and-Count(A,B)

维护一个**Current**指针指向每个表，初始化指向首元素

维护一个变量**Count**用于逆序的计数，初始为0

While两个表都不空

 令**a_i**与**b_j**是由**Current**指针指向的元素

 把这两个表中较小的元素加到输出表中

If **b_j**是较小的元素 **then**

 把**Count**加上在**A**中**剩余的元素数**

Endif

 把较小元素选出的表中的**Current**指针前移

Endwhile

一旦一个表为空，把另一个表剩余的所有元素加到输出中

返回**Count**和合并后的表

Sort-and-Count(L)

If这个表中有一个元素，**then**

 没有逆序

Else

 把这个表划分成两半，

A包含前 $\lceil n/2 \rceil$ 个元素

B包含剩下的 $\lfloor n/2 \rfloor$ 个元素

(rA,A)=Sort-and-Count(A)

(rB,B)=Sort-and-Count(B)

(r,L)=Merge-and-Count(A,B)

Endif

返回**r=rA+rB+r**以及排好序的表**L**

时间复杂度为 $O(n \log n)$

最邻近点对

可以证明，比较两侧的点的距离时，如果按照**y**值排序，每个点需要比较的点与他相距的位数不超过7

在计算分在两侧的点之间的距离时，如果采取按照**y**值排序，总的时间复杂度为 $O(n \log^2 n)$

如果每次递归都返回两个列表，一个是按照**x**排序，一个按照**y**排序，那么每次排序的时间复杂度就变成 $O(n)$ ，总时间复杂度变为 $O(n \log n)$

整数乘法

Recursive-Multiply(x,y)

令 $x = x_1 2^{n/2} + x_0, y = y_1 2^{n/2} + y_0$

计算 $x_1 + x_0$ 与 $y_1 + y_0$

$p = \text{Recursive-Multiply}(x_1 + x_0, y_1 + y_0)$

$x_1 y_1 = \text{Recursive-Multiply}(x_1, y_1)$

$x_0 y_0 = \text{Recursive-Multiply}(x_0, y_0)$

Return $x_1 y_1 2^n + (p - x_1 y_1 - x_0 y_0) 2^{n/2} + x_0 y_0$

时间复杂度为 $O(n^{\log_2 3}) = O(n^{1.585})$

矩阵乘积

- 问题的关键在于能否减少乘法的次数

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{aligned} P_1 &= A_{11} \times (B_{12} - B_{22}) \\ P_2 &= (A_{11} + A_{12}) \times B_{22} \\ P_3 &= (A_{21} + A_{22}) \times B_{11} \\ P_4 &= A_{22} \times (B_{21} - B_{11}) \\ P_5 &= (A_{11} + A_{22}) \times (B_{11} + B_{22}) \\ P_6 &= (A_{12} - A_{22}) \times (B_{21} + B_{22}) \\ P_7 &= (A_{11} - A_{21}) \times (B_{11} + B_{12}) \end{aligned}$$

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 \\ C_{12} &= P_1 + P_2 \\ C_{21} &= P_3 + P_4 \\ C_{22} &= P_5 + P_1 - P_3 - P_7 \end{aligned}$$

▣ 7 次乘法

▣ 18 = 10 + 8 次加减法

60

时间复杂度 $O(n^{\log_2 7}) = O(n^{2.81})$

卷积与快速傅里叶变换

目的：求多项式乘积

两个n阶多项式--FFT-->若干个点的乘积---IFFT-->2n-1阶多项式

$$e^{2k\pi i/n}$$

FFT算法

```
fft(n, a0, a1, ..., an-1) {  
    if (n == 1) return a0  
  
    (e0, e1, ..., en/2-1) ← FFT(n/2, a0, a2, a4, ..., an-2)  
    (d0, d1, ..., dn/2-1) ← FFT(n/2, a1, a3, a5, ..., an-1)  
  
    for k = 0 to n/2 - 1 {  
        ωk ← e2πik/n  
        yk ← ek + ωk dk  
        yk+n/2 ← ek - ωk dk  
    }  
  
    return (y0, y1, ..., yn-1)  
}
```

时间复杂度为 $O(n \log n)$

Ch6 动态规划

带权区间调度

分析

我们首先对区间 j 定义 $p(j)$ 为使得区间 i 与 j 不相交的最大的标记 $i < j$ ，也就是说， i 是最右边的与 j 不冲突的区间。

令 $Opt(j)$ 表示对于需求 $\{1, \dots, j\}$ 所组成的问题的最优解，我们寻找的最优解就是 $Opt(n)$ 。对于 $Opt(j)$ 可以分为以下两种情况：

- $Opt(j)$ 包含任务需求 j
 - 不会包含不相容的任务需求 $\{p(j) + 1, \dots, j - 1\}$
 - 包含剩下的任务需求 $1, 2, \dots, p(j)$ 的最优解
- $Opt(j)$ 不包含任务需求 j
 - 一定包含任务需求 $1, 2, \dots, j - 1$ 的最优解

于是有如下定理

定理1

$$OPT(j) = \begin{cases} 0 & , j = 0 \\ \max\{v_j + OPT(p(j)), OPT(j-1)\} & , j > 0 \end{cases}$$

其中 v_j 代表第 j 段区间所占权重

定理2：需求 j 属于集合 $\{1, 2, \dots, j\}$ 上的最优解当且仅当 $v_j + OPT(p(j)) \geq OPT(j-1)$

递归算法

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 

Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .

Compute  $p(1), p(2), \dots, p(n)$ 

Compute-Opt( $j$ )
{
    if ( $j = 0$ )
        return 0
    else
        return  $\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j-1))$ 
}
```

问题：在计算的过程中会出现冗余项；最坏情况下会以指数时间运行

动态规划算法

用数组存储中间计算结果

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 

Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .
Compute  $p(1), p(2), \dots, p(n)$ 

for  $j = 1$  to  $n$ 
     $M[j] = \text{empty}$   $\leftarrow$  全局数组
 $M[0] = 0$ 

M-Compute-Opt( $j$ ) {
    if ( $M[j]$  is empty)
    {
         $M[j] = \max(w_j + \text{M-Compute-Opt}(p(j)),$ 
                     $\text{M-Compute-Opt}(j-1))$ 
    }
    return  $M[j]$ 
}
```

定理3: M-Compute-Opt(n)的运行时间为 $O(n)$

动态规划时间复杂度

- 按照结束时间排序: $O(n \log n)$
- 计算全部 $p(\cdot)$: 如果已经对开始时间排序, 只需 $O(n)$
- 计算数组 M : $O(n)$

查找属于该调度的区间

应用定理2

```
Run M-Compute-Opt(n)
Run Find-Solution(n)

Find-Solution(j) {
    if (j = 0)
        output nothing
    else if ( $v_j + M[p(j)] > M[j-1]$ )
        print j
        Find-Solution(p(j))
    else
        Find-Solution(j-1)
}
```

时间复杂度为 $O(n)$

分段最小二乘

定义

- $OPT(j)$ = 对于点 $p_1, p_2, \dots, p_j$ 的最优解
- $e(i, j)$ = 关系到 $p_i, p_{i+1}, \dots, p_j$ 的任何直线的最小误差
- c : 每次改变的罚分

定理1: 如果最优化分的最后一段是 $p_i, p_{i+1}, \dots, p_n$, 那么最优解的值是
 $OPT(n) = e(i, n) + c + OPT(i - 1)$

为了得到 $OPT(j)$, 对于某一个 i , 我们应该以最好的方式产生最后的一段 $p_i, p_{i+1}, \dots, p_j$

定理2: 对于 $p_1, p_2, \dots, p_j$ 的子问题

$$OPT(j) = \begin{cases} 0 & , j = 0 \\ \min_{1 \leq i \leq j} \{e(i, j) + c + OPT(i - 1)\} & , j > 0 \end{cases}$$

动态规划算法

INPUT: $n, p_1, \dots, p_N, c$

```
Segmented-Least-Squares() {  
    M[0] = 0  
    for j = 1 to n  
        for i = 1 to j  
            compute the least square error  $e_{ij}$  for  
                the segment  $p_i, \dots, p_j$   
  
    for j = 1 to n  
        M[j] = min  $_{1 \leq i \leq j} (e_{ij} + c + M[i-1])$   
  
    return M[n]  
}
```

时间复杂度分析

- 算法前半部分, 需要计算 $O(n^2)$ 个 $e(i, j)$, 计算每一个 $e(i, j)$ 需要 $O(n)$ 的复杂度, 因此前半部分的复杂度为 $O(n^3)$
- 算法后半部分, 总的代价是 $O(n^2)$

计算最优划分

- 通过数组M向回寻找计算**最优划分**

Find-Segments(j)

If $j=0$ then

不用输出

Else

找一个使得 $e(i, j) + c + M[i-1]$ 最小的 i

输出这一段 $\{p_i, \dots, p_j\}$ 作为直线段, 以及输出递归函数Find-Segments($i-1$)的结果

End if

子集和与背包

有 n 个物品, 每一个物品都有一个值 v_i 和权 W , 需要在 $\sum_{i \in S} w_i \leq W$ 的情况下, 使 $\sum_{i \in S} v_i$ 最大化

定义

- $OPT(i, w)$ 表示使用物品 $1, \dots, i$ 的子集且所允许的最大权是 w 的最优解的值

于是存在两种情形

- OPT没有选择i: OPT选择子集 $\{1, 2, \dots, i-1\}$, 采用w的最优解
- OPT选择i: 余下部分OPT选择子集 $\{1, 2, \dots, i-1\}$, 采用 $w-w_i$ 的最优解

于是得到递推关系式

$$OPT(i, w) = \begin{cases} 0 & , j = 0 \\ OPT(i-1, w) & , w_i > w \\ \max\{OPT(i-1, w), v_i + OPT(i-1, w-w_i)\} & , j > 0 \end{cases}$$

由此可见, 增加一个变量重新进行动态规划, 会变成更小的子问题, 有助于设计动态规划算法。

算法

```
Input: n, w1, ..., wN, v1, ..., vN

for w = 0 to W
    M[0, w] = 0

for i = 1 to n
    for w = 1 to W
        if (wi > w)
            M[i, w] = M[i-1, w]
        else
            M[i, w] = max {M[i-1, w], vi + M[i-1, w-wi] }

return M[n, W]
```

时间复杂度分析

计算每一个M[i,w]的时间复杂度为 $O(1)$, 于是得到

定理: 上面的算法正确的计算这个问题的最优值, 并且运行在 $\Theta(nW)$ 时间

查找物品的最优集合

定理: 给定子问题的最优值的表M, 可以在 $O(n)$ 的时间内找到最优集合。

推广

运行时间不是n的多项式函数, 相反, 他是n与W的多项式函数, 我们把这个算法称为伪多项式的。

RNA的二级结构

配对要求

- 满足碱基互补原则
- 没有尖的转弯, 即如果 $(b_i, b_j) \in S$, 那么 $i < j - 4$
- 不交叉, 即如果 $(b_i, b_j), (b_k, b_l) \in S$, 那么不能有 $i < k < j < l$
- 没有碱基出现在一个以上的对中

需要求解具有最大碱基配对个数的RNA的二级结构

定义 $OPT(i, j)$ 表示在 $b_i b_{i+1} \cdots b_j$ 上的二级结构碱基对的最大数目，有以下情形

- 如果 $i \geq j - 4$, $OPT(i, j) = 0$
- 如果碱基 b_j 没有配对, $OPT(i, j) = OPT(i, j - 1)$
- 如果碱基 b_j 与碱基 b_t 配对, 其中 $i \leq t \leq j - 4$,

$$OPT(i, j) = 1 + \max_{i \leq t \leq j-4} \{OPT(i, t-1) + OPT(t+1, j-1)\}$$

于是有

定理:

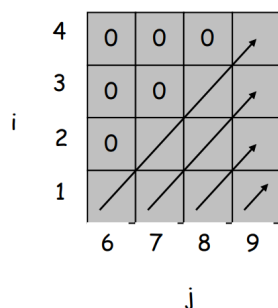
$$OPT(i, j) = \max(OPT(i, j-1), 1 + \max_{i \leq t \leq j-4} (OPT(i, t-1) + OPT(t+1, j-1)))$$

算法

```
RNA( $b_1, \dots, b_n$ ) {  
  for  $k = 5, 6, \dots, n-1$   
    for  $i = 1, 2, \dots, n-k$   
       $j = i + k$   
      Compute  $M[i, j]$   
  
  return  $M[1, n]$   
}
```

↖
using recurrence

注意此问题的填表格的顺序是按照区间长度增长顺序，即按照表格的斜对角线填入。



时间复杂度

存在 $O(n^2)$ 个子问题，每次求值用 $O(n)$ 时间，总运行时间为 $O(n^3)$

序列匹配

定义字母与空隙罚分 δ ，两个字母不匹配罚分 α_{pq} ，总罚分等于空隙罚分与不匹配罚分之和。

定理：在一个最优比对 M 中，至少下述情况之一为真：

1. $(m, n) \in M$

2. X的第m个位置没被匹配

3. Y的第n个位置没被匹配

现在定义 $OPT(i, j) = x_1x_2 \cdots x_i$ 与 $y_1y_2 \cdots y_j$ 比对的最小罚分, 分为以下三种情况

- 比对 x_iy_j 在OPT中, 这时的罚分为: $\alpha_{x_iy_j} + OPT(i-1, j-1)$
- OPT中 x_i 没有匹配, 这时的罚分为: $\delta + OPT(i-1, j)$
- OPT中 y_j 没有匹配, 这时的罚分为: $\delta + OPT(i, j-1)$

所以OPT(i,j)的递归式为

$$OPT(i, j) = \begin{cases} j\delta & , i = 0 \\ i\delta & , j = 0 \\ \min \begin{cases} \alpha_{x_iy_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases} & , i \geq 1, j \geq 1 \end{cases}$$

于是有如下定理

对于 $i, j \geq 1$,

$OPT(i, j) = \min[\alpha_{x_iy_j} + OPT(i-1, j-1), \delta + OPT(i-1, j), \delta + OPT(i, j-1)]$, 此外 (i, j) 在关于这个子问题的一个最优比对 M 中, 当且仅当这个最小值是由上式中的第一个值达到的

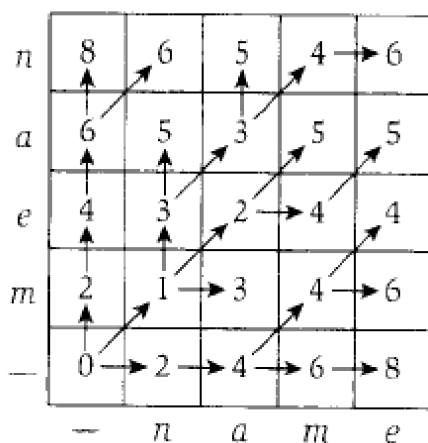
算法

```
Sequence-Alignment(m, n, x1x2...xm, y1y2...yn,  $\delta$ ,  $\alpha$ ) {  
    for i = 0 to m  
        M[0, i] = i $\delta$   
    for j = 0 to n  
        M[j, 0] = j $\delta$   
  
    for i = 1 to m  
        for j = 1 to n  
            M[i, j] = min( $\alpha[x_i, y_j] + M[i-1, j-1]$ ,  
                         $\delta + M[i-1, j]$ ,  
                         $\delta + M[i, j-1]$ )  
    return M[m, n]  
}
```

时间和空间复杂度为 $O(mn)$

分析算法

- 建立一个二维方格图 G_{xy} , 用来表示状态之间的转换。
- 例子：间隙罚分 $\delta = 2$, 元音与不同元音配对，辅音与不同辅音配对不匹配罚分1；元音与辅音配对罚分3。



83

定理：令 $f(i, j)$ 表示 G_{xy} 中从 $(0, 0)$ 到 (i, j) 的一条路径最小的代价，那么对于所有的 i, j ，我们有 $f(i, j) = OPT(i, j)$

线性空间的序列比对

可以将时间复杂度降低到 $O(m)$

Space-Efficient-Alignment (X, Y)

数组 $B[0 \dots m, 0 \dots 1]$

初始化对每个 i 令 $B[i, 0] = i\delta$

For $j=1, \dots, n$

$B[0, 1] = j\delta$

For $i=1, \dots, m$

$B[i, 1] = \min[\alpha_{x_j y_j} + B(i-1, 0), \delta + B(i-1, 1), \delta + B(i, 0)]$

Endfor

将B的第一列移到第0列来为下一次迭代留出空间

对每个 i 修改 $B[i, 0] = B[i, 1]$

Endfor

但是这样无法恢复比对本身。下面可以使用空间复杂度为 $O(m + n)$ 的方法求解

将前面的二维网格逆向看，定义 $g(i, j)$ 是在 G_{xy} 中从 (i, j) 到 (m, n) 的最短路径长度，那么

定理：对于 $i < m$ 与 $j < n$ ，我们有

$g(i, j) = \min[\alpha_{x_{i+1} y_{j+1}} + OPT(i+1, j+1), \delta + OPT(i, j+1), \delta + OPT(i+1, j)]$

将正向与逆向公式结合，可以得到如下结论

命题：在 G_{xy} 中通过 (i, j) 的角到角的最短路径的长度是 $f(i, j) + g(i, j)$ 。

于是可以得到如下定理

定理：令 k 表示在 $\{0, \dots, n\}$ 中的任何数，且令 q 是使得量 $f(q, k) + g(q, k)$ 达到最小的序标，那么存在一条通过结点 (q, k) 的最小长度的角到角的路径。

基于该定理，使用动态规划算法和分治策略可以在空间复杂度为 $O(m + n)$ 的情况下得出得到比对并得到最小罚分

- 划分：采用动态规划算法，寻找 q 使得 $f(q, n/2) + g(q, n/2)$ 最小
- 处理：维护全局可访问的表来保存中间结点

Divide-and-Conquer-Alignment (X, Y)

令 m 是 X 中的符号个数

令 n 是 Y 中的符号个数

If $m \leq 1$ 或 $n \leq 2$, then

使用Alignment (X, Y) 计算最优比对

调用Space-Efficient-Alignment (X, Y[1:n/2])

调用Backward-Space-Efficient-Alignment (X, Y[n/2+1:n])

令 q 是使得 $f(q, n/2) + g(q, n/2)$ 达到最小的指标

把 $(q, n/2)$ 加到全局表 P 中

Divide-and-Conquer-Alignment (X[1:q], Y[1:n/2])

Divide-and-Conquer-Alignment (X[q:m], Y[n/2:n])

返回 P

98

该算法时间复杂度的递推关系式为

$$T(m, n) \leq cmn + T(q, n/2) + T(m - q, n/2)$$

$$T(m, 2) \leq cm$$

$$T(2, n) \leq cn$$

可以采用待定系数法求解。

定理：上述算法在长度为 m 和 n 的串上运行的时间是 $O(mn)$

图中的最短路径

只有在图中没有负圈的情况下考虑最小费用才有意义，所以下面集中考虑两个问题

- 给定一个带权图 G ，确定其是否有负圈
- 如果这个图没有负圈，找一条从始点 s 到终点 t 的路径 P 使得路径具有最小费用

Dijkstra算法对于有负权边的图无效。考虑将每条边的费用同时加上一个数，但是这样最后费用也和路径数量有关

对于有负权边的图，使用Bellman-Ford算法。基于如下定理

定理：如果 G 没有负圈，那么存在一条从 s 到 t 的简单的最短路径（即没有重复节点），因此它至多只有 $n - 1$ 条边。

使用 $OPT(i, v)$ 表示至多使用 i 条边的 v - t 最短路径的最小费用，我们的目的是计算 s 到 t 的最短路径：

$OPT(n - 1, s)$

- 如果路径 P 至多使用 $i - 1$ 条边，那么 $OPT(i, v) = OPT(i - 1, v)$
- 如果路径 P 使用 i 条边，并且第一条边是 (v, w) ，那么余下的是 w - t 路径至多使用 $i - 1$ 条边：

$$OPT(i, v) = c_{vw} + OPT(i - 1, w)$$

定理：如果 $i > 0$ ，那么

$$OPT(i, v) = \min(OPT(i - 1, v), \min_{w \in V} (OPT(i - 1, w) + c_{vw}))$$

算法

```
Shortest-Path( $G, t$ ) {  
    foreach node  $v \in V$   
         $M[0, v] \leftarrow \infty$   
     $M[0, t] \leftarrow 0$   
  
    for  $i = 1$  to  $n-1$   
        foreach node  $v \in V$   
             $M[i, v] \leftarrow M[i-1, v]$   
            foreach edge  $(v, w) \in E$   
                 $M[i, v] \leftarrow \min \{ M[i, v], M[i-1, w] + c_{vw} \}$   
}
```

时间复杂度

表 M 有 n^2 项，每项可以使用 $O(n)$ 时间计算，于是

定理：在任何没有负圈的图中，上述算法正确的计算了一条 s - t 路路径的最小费用，并且运行在 $O(n^3)$ 时间

寻找最短路径的方法：每次记录后继元素

改进

定理：前面的Shortest-Path算法可以改进到 $O(mn)$ 时间

可以给出运行时间的界 $O(n \sum_{v \in V} n_v)$ ， n_v 为从结点 v 的结点个数，于是 $\sum_{v \in V} n_v = m$ ，于是时间复杂度为 $O(mn)$

对于存储，可以对每个结点 v 更新一个值 $M(v)$ ，即我们至今已经找到的从 v 到 t 的最短路径长度

命题：在整个算法中， $M[v]$ 是从 v 到 t 的某条路经的长度，且在 i 轮更新后 $M[v]$ 的值不大于从 v 到 t 至多使用 i 条边的最短路径长度

```

Push-Based-Shortest-Path( $G, s, t$ ) {
  foreach node  $v \in V$  {
     $M[v] \leftarrow \infty$ 
     $successor[v] \leftarrow \phi$ 
  }

   $M[t] = 0$ 
  for  $i = 1$  to  $n-1$  {
    foreach node  $w \in V$  {
      if ( $M[w]$  has been updated in previous iteration) {
        foreach node  $v$  such that  $(v, w) \in E$  {
          if ( $M[v] > M[w] + c_{vw}$ ) {
             $M[v] \leftarrow M[w] + c_{vw}$ 
             $successor[v] \leftarrow w$ 
          }
        }
      }
    }
    If no  $M[w]$  value changed in iteration  $i$ , stop.
  }
}

```

为了复原这条路径，记录每个结点通向终点 t 的路径上它后面的第一个结点；只要距离 $M[v]$ 更新，我们就更新这个值($first[v]$)

现在令 P 表示这个有向“指针图”，他的结点时 V 且边是 $\{(v, first[v])\}$ ，那么

命题：如果指针图 P 包含一个圈 C ，那么这个圈一定有负费用

命题：假设 G 没有负圈，考虑算法终止时的指针图 P ，对每个结点 v ，在 P 中从 v 到 t 的路径是 G 中的一条最短 v - t 路径

Ch7 网络流

一个 s - t 流需要满足两个条件

- 容量条件： $0 \leq f(e) \leq C_e$
- 守恒条件：除了 s, t 外，对每个结点 v ，满足 $\sum_{e_{in_v}} f(e) = \sum_{e_{out_v}} f(e)$

定义 $f^{out}(v) = \sum_{e_{out_v}} f(e)$, $f^{in}(v) = \sum_{e_{in_v}} f(e)$

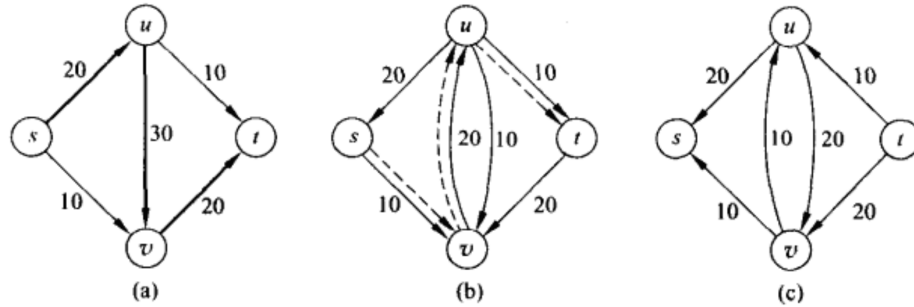
我们记 $v(f) = f^{out}(s)$

最大流问题

最大流问题：给定一个流网络，找出一个具有最大值的流。

剩余图

剩余图 给定流网络 G 以及 G 上的流 f , 我们如下定义 G 关于 f 的剩余图 G_f (见图 7.4, 这是在沿着路径 s, u, v, t 推 20 个单位的流以后, 关于图 7.3 的流的剩余图).



可以将剩余图中与原边方向相同的边理解为前向边, 与原边方向相反的边理解为后向边。

剩余图中的每一条边的容量都可以看成剩余容量, 且该容量大于零。

我们将上述的剩余图记为 G_f , 定义 P 是剩余图上的任意一条路径, 定义 $bottleneck(P, f)$ 是路径 P 上任何边关于流 f 的最小剩余容量, 我们现在定义下面的操作 $augment(f, P)$, 它在 G 中产生一个新的流 f' 。这里的 e 是路径 P 上的每一条边。

```
Augment(f, P) {
    b ← bottleneck(P)
    foreach e ∈ P {
        if (e ∈ E) f(e) ← f(e) + b
        else      f(eR) ← f(e) - b
    }
    return f
}
```

前向边

后向边

通常把剩余图中的任何一条 s - t 路径认为是一条增广路径。

命题: f' 是 G 中的一个流

算法

使用如下算法来计算最大流, 称为 Ford-Fulkerson 算法

Max-Flow

```
对  $G$  中所有的  $e$  初始化  $f(e)=0$ 
While 在增广图  $G_f$  中存在一条  $s$ - $t$  路径
    令  $P$  是  $G_f$  中的一条简单路径
     $f' = \text{augment}(f, P)$ 
    把  $f$  更新为  $f'$ 
    把增广图  $G_f$  更新为  $G_{f'}$ 
Endwhile
Return  $f$ 
```

分析算法

命题：在Ford-Fulkerson算法的每个中间步，流值 $\{f(e)\}$ 和 G_f 中的剩余容量是整数

命题：令 f 是 G 中的流，且令 P 是 G_f 中的一条简单 s - t 路径，那么 $v(f') = v(f) + \text{bottleneck}(P, f)$ ；并且由于 $\text{bottleneck}(P, f) > 0$ ，我们有 $v(f') > v(f)$

定理：假设在流网络 G 中的所有容量都是整数，那么Ford-Fulkerson算法在至多 C 次While循环的迭代后终止

上面的 C 是从始点 s 流出的流之和的最大值，即 $v(f) \leq C = \sum_{e \text{ 从 } s \text{ 出来}} c_e$

时间复杂度

定理：假设在流网络 G 中的所有容量都是整数，那么Ford-Fulkerson算法的运行可以在 $O(mC)$ 时间内实现

剩余图至多有 $2m$ 条边，查找路径的代价为 $O(m + n) = O(m)$ ，建立新的剩余图要 $O(m)$ 时间，while循环至多 C 次，所以时间复杂度为 $O(mC)$

最大流与最小割

我们说一个 s - t 的割是结点集合 V 的一个划分 (A, B) ，使得 $s \in A, t \in B$ ，一个割 (A, B) 的容量记为 $c(A, B)$ ，也就是从 A 出来的所有边的容量之和

最小 s - t 割问题：寻找一个最小容量的 s - t 割

定理：令 f 是任何 s - t 流，且 (A, B) 是任意 s - t 割，那么 $v(f) = f^{out}(A) - f^{in}(A)$

命题：令 f 是任意的 s - t 流，且 (A, B) 是任意 s - t 割，那么 $v(f) = f^{in}(B) - f^{out}(B)$

定理：令 f 是任何 s - t 流，且 (A, B) 是任意 s - t 割，那么 $v(f) \leq c(A, B)$

推论：令 f 是任何 s - t 流，且 (A, B) 是任意 s - t 割，如果 $v(f) = c(A, B)$ ，那么 f 是最大流， (A, B) 是最小割

定理：如果 f 是使得剩余图 G_f 中没有 s - t 路径的一个 s - t 流，那么在 G 中存在一个 s - t 割 (A^*, B^*) ，使得 $v(f) = c(A^*, B^*)$ 。因此， f 有 G 中任何流的最大值，且 (A^*, B^*) 有 G 中任何 s - t 割的最容量。

定理：给定一个最大值的流 $\bar{f}$ ，我们可以在 $O(m)$ 时间内计算一个最小容量的s-t割

命题：在每个流网络中，存在一个流 f 和一个割 (A, B) ，使得 $v(f) = c(A, B)$

定理：如果在流网络中所有的容量都是整数，那么存在一个最大流 f ，他的每个流值 $f(e)$ 都是整数

如果边的容量是实数，最大流最小割定理仍然成立，但是由于增广路径选择不合理，具有实数容量的Ford-Fulkerson算法可能永远运行下去

选择好的增广路径

如果每次选择的路径的瓶颈容量很小，收敛就会很慢，所以我们每次需要选择具有大的瓶颈容量的路径

为了便于控制，我们维护一个称为缩放参数的 Δ ，算法中将寻找瓶颈容量至少是 Δ 的路径

令 $G_f(\Delta)$ 是仅由剩余容量至少为 Δ 的边组成的剩余图的子集，我们将处理那些是2的幂的 Δ 值

```
Scaling-Max-Flow( $G, s, t, c$ ) {  
    foreach  $e \in E$   $f(e) \leftarrow 0$   
     $\Delta \leftarrow$  smallest power of 2 greater than or equal to  $C$   
     $G_f \leftarrow$  residual graph  
  
    while ( $\Delta \geq 1$ ) {  
         $G_f(\Delta) \leftarrow \Delta$ -residual graph  
        while (there exists augmenting path  $P$  in  $G_f(\Delta)$ ) {  
             $f \leftarrow$  augment( $f, c, P$ )  
            update  $G_f(\Delta)$   
        }  
         $\Delta \leftarrow \Delta / 2$   
    }  
    return  $f$   
}
```

算法分析

定理：如果容量是整数值，那么在缩放最大流算法中流和剩余容量也始终保持整数值，这就推出当 $\Delta = 1$ 时， $G_f(\Delta)$ 与 G_f 相同，因此当算法终止时，流 f 是最大值的流

命题：外层While循环的迭代次数至多是 $1 + \lceil \log_2 C \rceil$

命题：在 Δ 缩放阶段，每次增广增加的流值至少是 Δ

命题：令 f 是 Δ -缩放阶段结束时的流。 G 中存在一个s-t割 (A, B) 使得 $c(A, B) \leq v(f) + m\Delta$ ，其中 m 是图 G 中的边数，因此在网络中的最大流的值至多是 $v(f) + m\Delta$

命题：在一个缩放阶段增广次数至多是 $2m$

定理：在具有 m 条边和整数容量的图中，缩放最大流算法找最大流至多用 $2m(1 + \lceil \log_2 C \rceil)$ 次增广，它可以实现在至多 $O(m^2 \log_2 C)$ 时间内运行。

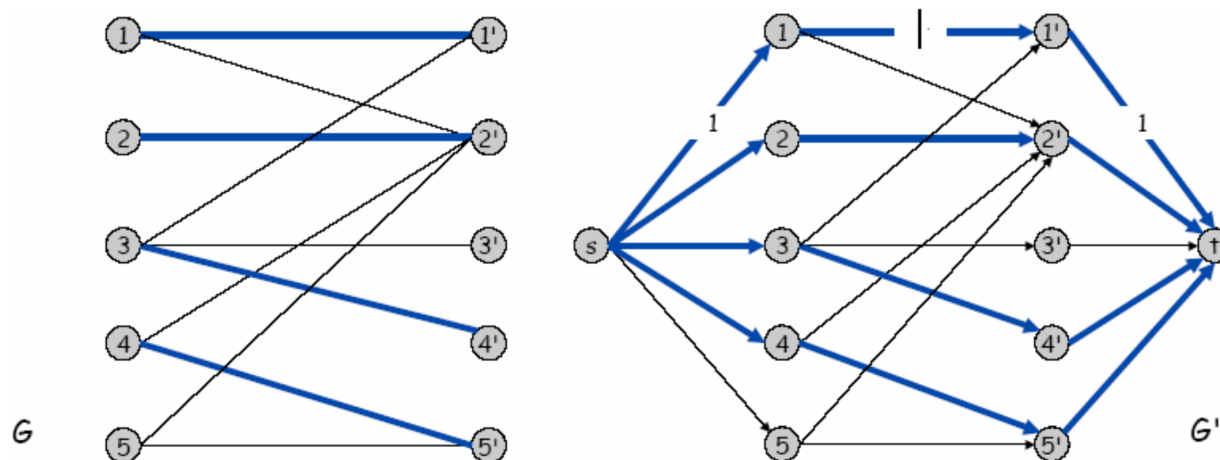
推广：强多项式算法

每次迭代选择具有最少边数的增广路径，时间复杂度为 $O(mn)$

二分匹配问题

在二部图中，寻找具有最大边数的二分匹配问题，其中每个顶点之出现在一条边中

构造图 G' ，他在原来的二部图 G 的基础上增加了源点 s ，终点 t ，源点 s 与二部图的L部分中的每个点相连，终点 t 与二部图R部分中的每个点相连， G' 中的每条边赋容量为单位容量1



定理：G中最大匹配数目与所定义的 G' 中最大流值相同

定理：可以使用Ford-Fulkerson算法在 $O(mn)$ 时间内找到二部图中的一个最大匹配

有向与无向图中的不交路径

定义两条路径是不交路径，如果他们没有任何公共边。

我们定义一个流网络，每条边有容量1。假设存在 k 条边不相交的 s - t 路径，我们可以使每一条这样的路径携带一个单位的流

对任何这种路径的边 e 设定 $f(e) = 1$ ，其他的边 $f(e') = 0$

有向图的不交路径

命题：如果在有向图 G 中从 s 到 t 存在 k 条边不交的路径，那么 G 中的最大 s - t 流至少是 k 。

命题：如果 f 是值为 v 的0-1流，那么具有流值 $f(e) = 1$ 的边的集合包含一组 v 条（边不交）路径。

定理：在有向图 G 中从 s 到 t 存在 k 条边不交路径当且仅当在 G 中 s - t 流的最大值至少是 k 。

定理：可以使用Ford-Fulkerson算法在 $O(mn)$ 时间内在有向图 G 中找到一组最大的边不交 s - t 路径。

相关问题

如果从图 G 中拿走一组边后图中一条 s - t 路径也没剩下，我们就说 F 分离 s 与 t

定理：在每个具有结点 s 与 t 的有向图中，边不交 s - t 路径的最大数目等于为分离 s 与 t 所需移走的最少边数

无向图的不交路径

将图中的每条边以两个反向的有向边代替

注意两条路径在有向图中可能是边不交的，但是在无向图中可能共享一条边

但是在任何网络中总存在一个最大流，它在每一对相反的有向边中至多使用一条

命题：在任何流网络中存在一个最大流 f ，其中对所有相反的有向边 $e(u, v)$ 与 $e' = (v, u)$ ，有 $f(e) = 0$ 或 $f(e') = 0$ ，如果流网络的容量是整数，那么也存在这样一个整数的最大流。

定理：在无向图 G 中在 k 条从 s 到 t 的边不交的路径，当且仅当在 G 对应的有向图 G' 中 s - t 流的最大值至少是 k 。此外可以用Ford-Fulkerson算法在 $O(mn)$ 时间内找到无向图 G 中的一组最大的边不交的 s - t 路径。

定理：在每个具有结点 s 与 t 的无向图中，边不交的 s - t 路径的最大数目等于为分离 s 与 t 所需要移走的最少边数

Ch8 NP问题与计算的难解性

多项式时间归约

如果对于问题 Y ，能够使用多项式个标准的计算步骤，加上多项式次调用解问题 X 的黑盒子来解决问题 Y ，那么记作 $Y \leq_p X$ ，读作“ Y 多项式时间可归约到 X ”

定理：假设 $Y \leq_p X$ ，如果 X 能在多项式时间内求解，则 Y 也能在多项式时间内求解

定理：假设 $Y \leq_p X$ ，如果 Y 不能在多项式时间内解决，则 X 不能在多项式时间内解决

规约的基本策略

- 简单等价规约
- 从特殊情形规约
- 使用零件规约

独立集 顶点覆盖 集合覆盖 集合包装

在图 $G(V, E)$ 中，如果顶点集合 $S \subseteq V$ 中的任意两点之间没有边，则称 S 是独立的

独立集问题：给定图 G 和数 k ，问 G 包含大小至少为 k 的独立集吗

在图 $G(V, E)$ 中，如果每一条边 $e \in E$ 至少有一个端点在 S 中，则称 S 是一个顶点覆盖

顶点覆盖问题：给定图 G 和数 k ，问 G 包含大小至多为 k 的顶点覆盖吗

引理：设 $G=(V,E)$ 是一个图， $S \subseteq V$ ，那么 S 是一个独立集当且仅当他的补图 $V-S$ 是一个顶点覆盖

定理：独立集 $\leq_q$ 顶点覆盖

集合覆盖：试图用一组较小的集合覆盖一个任意的对象集合

定理：顶点覆盖 $\leq_q$ 集合覆盖

集合包装问题：把大量集合包装在一起，限制他们中的任意两个都不重叠

定理：独立集 $\leq_q$ 集合包装

使用“零件”归约

定理: $3\text{-SAT} \leq_q \text{独立集}$

规约的传递性

定理: 如果 $X \leq_q Y$ 且 $Y \leq_q Z$, 则 $X \leq_q Z$

于是有 $3\text{-SAT} \leq_q \text{独立集} \leq_q \text{顶点覆盖} \leq_q \text{集合覆盖}$

这些都是NP完全问题

有效证书和NP定义

NP是所有存在有效验证程序的问题的集合

P: 存在多项式解法的问题集合

定理: $P \subseteq NP$

定理: 如果Y是一个NP完全问题, X属于NP且 $Y \leq_q X$, 则X是NP完全的

NP完全问题

电路可满足性是NP完全的

NP完全问题

- 难问题的部分分类
 1. 包装问题: 独立集, 集合包装
 2. 覆盖问题: 顶点覆盖, 集合覆盖
 3. 划分问题: 三维匹配, 图着色
 4. 排序问题: 哈密尔顿圈, 哈密尔顿回路, 巡回售货员问题
 5. 数值问题: 子集和, 带开放时间和截止时间的调度问题
 6. 约束满足问题: SAT, 3SAT
- 一般经验: NP问题要么是P, 要么是NP完全问题